Datadog Pipeline Monitoring User Manual

A practical guide for pipeline developers using Datadog with Microsoft Fabric pipeline monitoring.

1. Overview

This guide explains how to send pipeline lifecycle events and table-level metadata from a Microsoft Fabric pipeline into Datadog for operational visibility. It is written for developers who need to understand the pipeline logic, configure monitoring correctly, and use the pattern in real workloads.

The pattern is intentionally simple:

- a parent pipeline starts
- a child or business pipeline executes
- a monitoring sender emits a Started, Success, or Error status
- the payload includes pipeline metadata and optional table metrics
- Datadog receives a structured JSON event that can be searched and filtered

This document focuses on the user-facing operational model rather than repository or deployment history.

2. What the pipeline does

The monitoring flow has three basic stages:

1. Started notification
    - Sent when the business workflow begins.
    - Captures run context such as pipeline name, run ID, trigger type, and start time.
2. Business execution
    - Runs the actual pipeline or task being monitored.
    - If it succeeds, the framework generates a success notification.
    - If it fails, the framework generates an error notification.
3. Final status notification
    - Success or Error is emitted after the business process completes.
    - The payload includes a clear business status and may also include extracted table metrics.

The design is intended to support operational observability, not to replace Fabric monitoring itself. It provides a clean, searchable signal in Datadog for pipeline health, execution timing, data quality context, and failure diagnostics.

3. Pipeline logic and execution flow

The monitoring process usually follows this flow:

- Start event is sent.
- Business pipeline is invoked.
- If the business pipeline succeeds, a success event is sent.
- If the business pipeline fails, an error event is sent.
- Optional table metric extraction runs and adds row counts and timestamp/source details.
- Datadog receives the final JSON payload.

In practical terms, the sender pipeline reads runtime values such as:

- pipeline name
- run ID
- trigger type
- trigger timestamp
- environment label
- service label
- table and schema names when needed
- optional error message

The sender then builds a JSON body and sends it using an authenticated HTTP call to the Datadog endpoint.

Standard statuses

The payload status values are typically:

- Started
- Success
- Error

The sender usually translates these to a Datadog-friendly status field such as:

- info for Started
- ok for Success
- error for Error
- warn for unusual or unexpected states

The important rule is consistency: the same parent pipeline values should be passed through all lifecycle messages for accurate correlation.

4. Required parameters

The sender is designed to receive a consistent set of runtime values. The most important parameters are listed below.

Parameter	Purpose	Recommended value
pipeline_status	Lifecycle state of the pipeline	Started, Success, or Error
schema_name	Schema of the target table	For example, dbo
table_name	Table being monitored	For example, companies
refresh_type	Business load label	Full Load or Incremental Load
start_time_utc	Trigger start time	@pipeline().TriggerTime
pipeline_name	Parent pipeline identity	@pipeline().PipelineName
run_id	Correlation key for the run	@pipeline().RunId
trigger_type	Trigger source	@pipeline().TriggerType
err_msg	Error details for failures	Empty for Started and Success

Guidance

- Use the same run_id across Started, Success, and Error for a single execution.
- Use exact case-sensitive status values such as Started and Success.
- Supply error details only on the Error path.
- Keep schema_name and table_name populated when the sender is expected to emit table metrics.

5. Payload structure

The payload follows a log-style JSON structure with metadata, status fields, and custom parameters. A representative example looks like this:

```
JSON
{
```

```
  "message": "Fabric Pipeline executed successfully.",
  "ddsource": "microsoft-fabric",
  "service": "data-ingestion-pipeline",
  "pipeline_status": "Success",
  "status": "ok",
  "pipeline_metadata": {
    "workspace_name": "fabric-dev",
    "pipeline_name": "sales-load-pipeline",
    "run_id": "abc123",
    "trigger_type": "Manual",
    "start_time_utc": "2026-10-12T10:00:00Z",
    "end_time_utc": "2026-10-12T10:05:00Z"
  },
  "custom_parameters": {
    "environment": "dev",
    "schema_name": "dbo",
    "table_name": "companies",
    "refresh_type": "Full Load",
    "row_count": 12458,
    "max_dw_updated": "2026-10-12T09:58:00Z",
    "min_dw_updated": "2026-10-12T09:00:00Z",
    "source": "crm"
  }
}
```


Key fields

Field	**Meaning**
message	Human-readable operational message
ddsource	Source system label
service	Logical service name in Datadog
pipeline_status	Business status value
status	Datadog-friendly status classification
pipeline_metadata	Runtime context for correlation
custom_parameters	Business and data details
error	Error message and monitoring link when applicable

6. Table metrics and enrichment

When configured, the monitoring sender can also collect table-level metrics from a notebook or query step. This is useful for validating that a load succeeded and for adding operational context.

Typical values included in the payload are:

- row_count
- max_dw_updated
- min_dw_updated
- source
- schema_name
- table_name
- refresh_type

How the metrics are used

These fields help answer real developer questions such as:

- How many rows were processed?
- Which table was affected?
- What was the latest update timestamp?
- Was the source value expected?
- Did the pipeline run in a full or incremental refresh?

This data is especially helpful when the pipeline fails or when a batch has suspiciously low or high row counts.

7. Setup and configuration

1. Create the monitoring sender

Use a dedicated pipeline whose responsibility is to:

- fetch the Datadog API key from a secure secret source
- receive parent execution metadata
- run the metrics notebook if table metrics are required
- build the final JSON payload
- post it to Datadog

2. Secure the secret

The API key must not be hardcoded. Load it from a secure secret manager or Key Vault and inject it at runtime only.

Best practices:

- store the key in a secure secret store
- restrict access to the monitoring pipeline identity
- use secure output/input settings when supported
- never log the API key in pipeline output or notebook stdout

3. Configure Datadog destination

Confirm the final HTTP destination and authentication method before production use.

Verify:

- Datadog site or region
- correct API endpoint
- correct header values
- proper content type
- connectivity from the Fabric environment

4. Validate labels

Set environment-specific labels carefully. Avoid copying development labels to production. Common labels include:

- environment
- service
- workspace_name
- pipeline_name

These labels are important for filtering Datadog logs and dashboards.

8. How to use it in practice

Follow this pattern for each pipeline you want to monitor.

Recommended workflow

1. Start the parent pipeline and capture runtime context.
2. Invoke the business pipeline or job.
3. Send a Started message with run metadata.
4. On completion, send either Success or Error.

5. Include table metrics only when relevant.
6. Use the same run_id across all lifecycle events.
7. Validate the payload in Datadog Logs or metrics views.

Example usage pattern

Use these values for a real pipeline run:

- pipeline_status: Started on start event
- pipeline_status: Success on successful completion
- pipeline_status: Error on failure
- schema_name: dbo
- table_name: companies
- refresh_type: Full Load
- start_time_utc: @pipeline().TriggerTime
- pipeline_name: @pipeline().PipelineName
- run_id: @pipeline().RunId
- trigger_type: @pipeline().TriggerType
- err_msg: empty string for Started and Success; actual failure message for Error

This keeps one execution correlated across all events and gives Datadog a consistent operational view.

9. Validation checklist

Before using the pattern in production, confirm the following:

- the secret can be retrieved successfully
- the Datadog endpoint is reachable from Fabric
- the API key is valid and not exposed in logs
- pipeline_status values are exact and consistent
- run_id is stable across the whole execution
- table metrics are present when expected
- error messages are relevant and sanitized
- Datadog shows the event under the right service and environment

Good validation flow

- run a normal successful execution
- run a deliberate failure case
- verify Started, Success, or Error appears with the correct metadata
- confirm logs are searchable by service, pipeline name, and run_id
- confirm the payload contains the expected custom parameters

10. Troubleshooting

No Datadog event appears

Check the following:

- API key retrieval succeeded
- HTTP request reached the expected endpoint
- connection configuration is correct
- pipeline status values are valid
- the sender pipeline executed end-to-end

Event is sent but missing metadata

Check:

- run_id propagation
- pipeline_name mapping
- trigger_type values
- start_time_utc value
- schema_name and table_name values

Table metrics are blank

Check:

- target table exists
- schema and table names are valid
- notebook has access to the source lakehouse or table
- the notebook output is being parsed correctly
- the query did not fail silently

Error status is unclear

Check:

- whether the error message was passed correctly
- if the business pipeline failed before the status call executed
- whether the payload includes a useful message and run correlation

Bad payload or rejected JSON

Avoid inserting unescaped content into JSON strings. Pay special attention to:

- quotes

- backslashes
- line breaks
- control characters
- freeform error messages

A controlled payload should be cleaned before sending to ensure Datadog accepts it.

11. Best practices

- Use one consistent run_id across the full pipeline execution.
- Keep all lifecycle events in the same service and environment namespace.
- Make the success and error messages clear and operationally useful.
- Include schema_name and table_name whenever business data context matters.
- Treat row_count and table metrics as evidence, not as proof that the full business process is complete.
- Restrict access to secrets and never expose sensitive values in pipeline logs.
- Validate both the payload shape and the actual Datadog record after deployment.

12. Summary

The Datadog monitoring pattern is most effective when it is treated as a developer-facing operational signal, not a side task. Used correctly, it gives pipeline developers a clear view of:

- when a pipeline started
- whether it succeeded or failed
- which run and trigger caused the event
- which table and dataset were involved
- what row and timestamp metrics were captured

This foundation makes troubleshooting faster, recovery easier, and pipeline health monitoring much more actionable in Datadog.